

Adaptation Experiment 4: TS-IFA

1 Final architecture

TS-IFA is a lightweight adapter around a frozen forecast backbone F . All quantities below are expressed after query-scale instance normalization. For a query history $x \in \mathbb{R}^L$, retrieved histories $x_j^c \in \mathbb{R}^L$, retrieved futures $y_j^c \in \mathbb{R}^H$, and K neighbors, define

$$p = F(x), \quad p^c = F(x; \{x_j^c, y_j^c\}_{j=1}^K), \quad n_j = F(x_j^c), \quad e_j = y_j^c - n_j.$$

An optional transformed frozen expert uses past-only covariates:

$$\phi(x) = (\text{sgn}(x), \sqrt{|x|}), \quad p^\phi = F(x; \text{past covariates } \phi(x)).$$

No future value of ϕ is supplied. A backbone that ignores past covariates simply gives $p^\phi = p$. This extraction is disabled by default, so payloads created before this expert was introduced remain valid.

Independently, a learned transformed covariate may be enabled:

$$u_\psi = T_\psi(x) \in \mathbb{R}^H, \quad c_\phi = \tilde{p}^\phi + G_\phi([\tilde{p}^\phi, u_\psi]), \quad \tilde{p}^\phi = \begin{cases} p^\phi, & \text{if the precomputed expert is enabled,} \\ p, & \text{otherwise.} \end{cases}$$

Both T_ψ and G_ϕ are MLPs trained on T_1 . Thus the learned covariate can be studied without rerunning extraction; enabling both options uses the precomputed sign/root forecast as its anchor.

1.1 Learned branches

Residual attention retrieves errors from analogous *states*, not merely analogous histories:

$$q_r = [x, p], \quad k_j^r = [x_j^c, n_j], \quad v_j^r = e_j, \quad z_r = A_r(q_r, \{k_j^r\}, \{v_j^r\}) \in \mathbb{R}^H, \\ c_r = p + M_r([p, z_r]).$$

There is one query token and K neighbor tokens. Thus the attention produces one mixture over neighbors, not a separate attention distribution per forecast horizon.

The memory branch reads retrieved horizons directly:

$$z_m = A_m(x, \{x_j^c\}, \{y_j^c\}) \in \mathbb{R}^H, \quad c_m = W_m[p, z_m] + M_m([p, z_m]).$$

Unlike c_r , c_m is a direct forecast rather than a named residual correction. Its linear skip W_m and nonlinear MLP jointly learn a mixture of the vanilla forecast and retrieved horizons.

1.2 Candidate-conditioned, unconstrained rooter

The default four candidate forecasts are

$$(c_0, c_1, c_2, c_3) = (p, p^c, c_r, c_m).$$

If either transformed option is enabled, the fifth candidate is inserted:

$$(c_0, c_1, c_2, c_3, c_4) = (p, p^c, c_\phi, c_r, c_m),$$

The rooter is deliberately not given hand-designed distances, neighbor dispersion, or retrieval statistics. It learns its own retrieval state:

$$z_g = A_g(x, \{x_j^c\}, \{x_j^c\}) \in \mathbb{R}^D.$$

Each active candidate has a learned type token $\tau_j \in \mathbb{R}^D$. One MLP S_θ , shared across candidates, produces an entire horizon-wise coefficient vector:

$$a_j = S_\theta(\text{LN}(z_g), \text{LN}(p), \text{LN}(c_j), \text{LN}(\tau_j)) \in \mathbb{R}^H,$$

$$\hat{y} = p + \sum_{j=0}^{J-1} a_j \odot c_j, \quad J \in \{4, 5\}.$$

The coefficients are unconstrained: there is no sigmoid, softmax, or convexity constraint. This retains the extrapolation ability observed for ridge baselines. The vanilla fallback is $a_j = 0$ for every j . A direct fallback to candidate c_j is representable by $a_0 = -1$, $a_j = 1$, and all other coefficients zero.

2 Initialization and optional regularization

With vanilla-anchoring initialization enabled,

$$M_r^{\text{out}} = 0, \quad W_m = [I_H \ 0], \quad M_m^{\text{out}} = 0, \quad G_\phi^{\text{out}} = 0, \quad S_\theta^{\text{out}} = 0.$$

Consequently $c_r = c_m = \hat{y} = p$ exactly at step zero; when the learned transformed branch is enabled, $c_\phi = \tilde{p}^\phi$. This initialization is optional; disabling it restores the standard PyTorch initialization.

The branches are trained on T_1 with

$$\mathcal{L}_B = \ell_n(c_r, y) + \ell_n(c_m, y) + \mathbb{I}_{\text{learn-}\phi} \ell_n(c_\phi, y) + \lambda_V \{\ell_n(c_r, p) + \ell_n(c_m, p) + \mathbb{I}_{\text{learn-}\phi} \ell_n(c_\phi, p)\},$$

where ℓ_n is normalized MSE. After branch training, all branch weights are frozen. The neural rooter is trained on T_2 with

$$\mathcal{L}_R = \ell_n(\hat{y}, y) + \lambda_V \ell_n(\hat{y}, p) + \lambda_2 \frac{1}{JH} \sum_{j,h} a_{j,h}^2 + \lambda_S \frac{1}{J(H-1)} \sum_{j,h=1}^{H-1} (a_{j,h+1} - a_{j,h})^2.$$

The three regularizers are independent and optional: setting the corresponding coefficient to zero removes it. Defaults are $\lambda_V = \lambda_2 = \lambda_S = 10^{-2}$.

3 Separate ridge rooter

The frozen T_1 branches permit a rooter-only comparison on exactly the same candidate bank. A horizon-wise, no-intercept ridge rooter is fitted from exact T_2 sufficient statistics:

$$A_{\text{ridge}} = \arg \min_{A \in \mathbb{R}^{J \times H}} \sum_{i,h} \left[y_{i,h} - p_{i,h} - \sum_{j=0}^{J-1} A_{j,h} c_{i,j,h} \right]^2 + \alpha \|A\|_F^2.$$

Candidate columns are RMS-scaled before solving and mapped back afterward. The fit uses all selected T_2 examples and has exactly JH learned coefficients: $4H$ by default and $5H$ with a transformed candidate. It does not see T_3 .

4 Training and evaluation protocol

Extraction pools $T_1 + T_2$ and assigns the final 20% of whole query dates to T_2 . The staged pipeline is:

T_1 : train (c_r, c_m, c_ϕ) $\longrightarrow$ T_2 : fit ridge and neural rooters with branches frozen $\longrightarrow$ T_3 : evaluate once.

Each optimization epoch draws one random batch; deterministic full- T_2 diagnostics retain `-valid-eval-freq` and interval-average logs retain `-logging-eval-freq`. There is no T_3 checkpoint selection or refit. The neural and ridge rooters are deliberately trained independently, so the same branch checkpoint can support additional rooter experiments.

The launcher `ts_ifa.slurm` writes the T_1 branch checkpoint, neural checkpoint, ridge coefficients, histories, and complete T_3 predictions. Predictions are written batch by batch as separate NumPy arrays indexed by the sole current `prediction_manifest.json`; no aggregate prediction checkpoint is retained. `result_manifest.json` is written last as the completion marker. The metric artifact always reports vanilla, context, residual, and memory candidates, and reports transformed performance only when that candidate is active; it also reports the ridge-rooted system and full neural TS-IFA.

5 Dimensions and exact parameter count

The reference comparison uses $L = 512$, $H = 64$, $K = 15$, four attention heads, per-head dimension 32, hence $D = 128$, and 128-unit MLP hidden layers. K changes computation but not parameter count.

Component	Parameters
Residual attention A_r	363,328
Residual MLP M_r	24,768
Residual branch	388,096
Memory attention A_m	346,944
Memory skip and MLP (W_m, M_m)	33,024
Memory branch	379,968
Both learned branches	768,064
Global rooter attention A_g	412,544
Shared scorer, layer norms, and four type tokens	58,688
Default neural rooter	471,232
Default full neural TS-IFA	1,239,296
Horizon-wise default ridge rooter only ($4H$)	256
Default branches plus ridge rooter	768,320
Precomputed transformed candidate: fifth type token	+128
Learned (T_ψ, G_ϕ) modules	+98,688
Full TS-IFA with learned transformed candidate	1,338,112

6 Parameter comparison

Counts below compare trainable adaptation parameters and exclude the frozen forecast backbone. Chronos-2 has 120M parameters and Chronos-Bolt-base 205M according to the [official Chronos model table](#). TS-RAG and Cross-RAG counts cover the added modules in their [official TS-RAG implementation](#) and [official Cross-RAG implementation](#).

Method	Trainable params.	$\times$ TS-IFA	% Chronos-2	% Bolt-base
TS-IFA, default	1,239,296	1.000	1.033	0.605
TS-RAG	4,775,425	3.853	3.980	2.329
Cross-RAG	8,712,192	7.030	7.260	4.250
Full horizon-wise ridge baseline	2,048	0.00165	0.00171	0.00100
Max. single CatBoost horizon gate	$\leq 384,000$	≤ 0.310	≤ 0.320	≤ 0.187
TS-IFA branches + ridge rooter	768,320	0.620	0.640	0.375

For $K = 15$, the unrelated full retrieval-ridge baseline has $H(2 + 2K) = 2,048$ coefficients. The CatBoost value is an upper-bound scalar complexity for 64 depth-4, 300-tree horizon models: $64 \cdot 300 \cdot (16 \text{ leaves} + 4 \text{ splits}) = 384,000$; early stopping can reduce it and tree structure is not directly equivalent to dense neural parameters.

7 Interpretation

Report every active candidate, ridge-rooted TS-IFA, and neural TS-IFA together. This separates candidate quality from rooter quality and tests whether the 471k-parameter conditional rooter earns its capacity over the 256-coefficient default ridge alternative (320 with the transformed candidate). Attention weights remain diagnostics rather than causal feature importance.